QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

TEAM ID: 128

TEAM NAME: NULL

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 - Team **Null**
Team Repo URL	https://github.com/mandy404-lgtm/EZ_System.git
Project Board URL	https://docs.google.com/document/d/1NReI-xE7okX3AecwQT0RmHreTuQOS5FCT7TCrzTy1mo/edit?usp=sharing
Live Deployment URL (optional for preliminary round)	-

Objective:

The primary objective of the EZ System is to process shop sales and inventory data to generate accurate, AI-driven business insights that help shop owners make better decisions. The system focuses on analyzing internal business data such as sales performance and stock levels, and uses the GLM-5.1 AI engine to produce recommendations like stock planning and pricing adjustments.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section ensures that every functional component of the **EZ System** is mapped to a specific test case, ensuring that the AI decision-making logic and CRUD operations fulfill the business requirements without feature creep.

1.1 In-Scope Core Features

These are the **must-test features** directly tied to your system requirements:

Home Page (Business Overview)

- KPI cards: total revenue, net profit, total cost
- AI business insight preview (latest GLM recommendation)

Product Page (CRUD)

- Create product (form validation, DB insert)
- Read product list (fetch from products table)
- Update product (edit fields, DB update)
- Delete product (remove record)
- Record sales

Analytics Page (AI Core)

- Run AI analysis
- Display AI recommendation, trade-off analysis, GLM reasoning
- Forecasts (revenue, demand, profit)
- Impact analysis (before vs after metrics)

Alerts Page

- Low stock alerts
- Low profit alerts
- High cost alerts
- Demand drop alerts

Profile Page

- Display business name, email
- Edit Profile
- Security Settings
- Help&Support
- Sign Out functionality

1.2 Out-of-Scope

- **Promotions & Offers**
- **Advanced Profile Management**
    - Uploading pictures

2. Risk Assessment & Mitigation Strategy

*[e.g. Quality Assurance risks are anticipatory. So, identify the technical risk associated with you application requirements, architecture. Now, the risk needs to be evaluated using the 5*5 Risk assessment Matrix. So, Risk Score = Likelihood * Severity.]*

Risk Assessment Scoring Criteria:

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
e.g., Payment Deducted but order status not updated	*3*	*5*	*15 (High)*	*Implement automatic rollback strategy to handle such failures. [e.g. Database transaction timeout increased to 10s]*	*Use manual/force failure after payment and monitor what gets triggered. [e.g. Sandbox Payment on Stripe]*

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
AI API request timeout or failure when generating product summary	4	4	16	Implement retry mechanism with exponential backoff and fallback cached summary	Simulate API failure using mock response or disable API endpoint
Database connection drops during insert/update	3	5	15	Enable connection pooling + retry + transaction rollback	Force DB disconnect during transaction
Login failure due to invalid or missing user input (e.g. empty fields, wrong email format, weak validation on frontend/backend)	3	3	9	Implement strict input validation using Pydantic (FastAPI) and frontend form validation to ensure correct data format before request	Test empty login fields, invalid email format, and missing password via UI and Postman
Login request failure due to server-side issues such as API timeout, database connection failure, or backend exception	2	5	10	Implement exception handling (try/except), database retry mechanism, and proper logging for debugging	Simulate DB shutdown, force API timeout, and send login request via Postman
Authentication failure due to incorrect credentials, expired JWT token, or invalid session handling	4	4	16	Use secure password hashing (bcrypt), JWT expiration handling, and proper token validation middleware	Test wrong password login, expired token access, and invalid JWT using Postman
Client sends invalid or malformed request causing	4	3	12	Implement strict input validation using Pydantic (FastAPI) and frontend form	Test with missing fields, invalid JSON, and wrong data types using Postman

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
400 Bad Request error (e.g. missing fields, wrong JSON format)				validation	
API endpoint or resource not found due to incorrect URL or missing database record	3	2	6	Ensure correct API routing, proper endpoint documentation, and validate database existence before query	Test invalid endpoints, deleted records, and wrong URLs using Postman
Unexpected server-side error causing system crash or failure	2	5	10	Implement proper exception handling, logging, and fallback mechanisms for API and database operations	Simulate DB failure, force exception in backend, and disable external API to test failure response
Client-side exception occurs due to invalid request, missing parameters, wrong data format, or frontend crash causing failed API calls	4	3	12	Implement strict input validation (Pydantic in FastAPI), frontend form validation, and error handling for API requests	Test empty inputs, invalid JSON payloads, incorrect data types, and broken request format using Postman and UI
Failure to fetch data from database causing frontend not to display required information	3	5	15	Implement proper API error handling, database connection pooling, retry mechanism, and fallback response (cached/default data)	Simulate database shutdown, API timeout, and invalid query response using Postman and backend testing

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*

4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**

  Scope:

  - Product CRUD logic (create, update, delete validation)
  - AI engine response parsing (GLM output cleanup, JSON formatting)
  - KPI calculations (profit = selling price − cost × units sold)

  Execution:

  - Written using the Dart test package for Flutter and PyTest for the Python backend.
  - Execution occurs locally during development and automatically via GitHub Actions upon every push.

  Isolation:

  - Database calls mocked using SQLite in-memory DB.
  - AI API calls mocked with sample JSON responses to test parsing logic.

  Pass Condition:

  - CRUD functions return correct DB changes.
  - AI parsing returns structured JSON (recommendation, trade-off, forecast).
  - Edge cases handled (e.g., stock = 0, negative profit).

- **Integration Test**

  Scope:

  Product Service ↔ Dashboard Service (KPI updates after CRUD).

  Product Service ↔ AI Service (analyze-product endpoint).

  Product Service ↔ Alerts Service (low stock triggers alert).

  Execution:

  Performed after Unit Testing is complete. This phase validates the "Activate AI Analyze" button flow where data moves from the products table to the AI service.

  Workflow: Tested in a **Staging Environment** using real API calls (no mocking) to ensure the Flutter app correctly parses the JSON output from the Z.AI GLM (Decision

Intelligence).

Pass Condition: The system must successfully display an AI recommendation card after a user clicks "Activate AI Analyze" on a specific product.

- **Test Environment (CI/CD Practice):**
  Local Testing: Manual testing performed on Flutter emulators and local Python development servers (Uvicorn/Flask).
  Staging/CI: GitHub Actions are configured to trigger automatically. Every time code is pushed to the EZ_SystemV2 repository, the system runs the build and test suite.
  CI/CD Automated Pipeline: * **Branch Level:** Basic syntax and linting checks.
    - **Pull Request (PR):** Full PyTest and Flutter test suites must pass before merging into the main branch.

- **Regression Testing & Pass/Fail Rules:**
  Execution Phase:
    - Regression tests run across all 5 in-scope features (Business Overview, Product CRUD, Analytics, Alerts, Profile) whenever new code merges to main.

  Pass/Fail Condition:

    - Test passes only if actual outcome = expected outcome.
    - Any mismatch logged in Defect Log.

  Continuation Rule:

    - Initial unit tests must pass before integration tests run.
    - Integration must pass before regression tests proceed.

Test Data Strategy:
  - **Manual:** Created 10 Mock SME Profiles containing varied inventory data (e.g., one business with high stock/low sales, another with low stock/high demand) to test GLM reasoning diversity.
  - **Automated:** SQL Seed Scripts are used to populate the products and ai_product_summary tables with 50+ dummy records to test the AI Business Insights on Analytic page

- **Passing Rate Threshold:**
  Minimum Passing Rate:
    - 86.2% of all test cases must pass.
  Critical Tests:

  100% pass required for:

    - Product CRUD
    - AI Analysis endpoint

- Alerts (low stock, low profit)

No exceptions allowed

4. CI/CD Release Thresholds & Automation Gates

This section defines the **quality gates used in the CI/CD pipeline**. These thresholds ensure that only stable, secure, and validated code is allowed to progress from development to production. If any condition fails, the pipeline will automatically block deployment.

4.1 Integration Thresholds (Merging to Main)

Check	Requirement	Project Result	Status
Build	Zero build errors	5 errors	Failed
Unit Tests	100% pass rate	100%	Passed
Code Quality	Zero linting errors	97%	Failed
Test Coverage	≥ 81.2%	79%	Failed
API Integration	All endpoints working	Partial failure	Failed
Database Integration	CRUD successful	Passed	Passed
AI Integration	Output correctly processed	Below threshold	Failed

4.2 Deployment Thresholds (Pushing to Production)

Check	Requirement	Project Result	Status	Interpretation
Regression Test	Minimum 90% pass rate	91%	Passed	Existing functionalities remain stable after updates

AI Output Pass Rate	≥ 80% accuracy based on documented prompts	78%	Failed	AI responses are below acceptable reliability level
Critical Bugs	Zero P0/P1 bugs allowed	2	Failed	Presence of critical issues makes system unsafe for production
API Performance	Response time < 800ms	450ms	Passed	System performance is within acceptable limits
Security	No exposed API keys or sensitive data	Clean	Passed	No security vulnerabilities detected

5. Test Case Specifications (Drafts)

This section is for you to draft your test cases. The test cases MUST include at least:

- ***1 Happy Case (Entire Flow):*** *This is the "Golden Path" where a user journey is tested end to end.*
- ***1 Specific Case (Negative/Edge Test):*** *This case should portray a scenario which triggers error in the system and the system handles it with proper handling techniques.*
- **2 Non-Functional (NFR) Tests (Performance/Load)*****:*** *This test consists of non-feature-based tests but architectural tests.*

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	**Happy Case (Entire Flow):** E-commerce AI Analysis Pipeline	Verify that a user can successfully perform an end-to-end workflow including logging in, managing product data, triggering AI analysis, and viewing generated insights. This ensures all system modules (API, database, AI engine, KPI calculation) are functioning together correctly.	1. Login to the system with valid credentials 2. Add a new product with cost price, selling price, and category 3. Save product into the database 4. Trigger AI analysis for the selected product 5. System processes reviews and generates AI summary 6. KPI (e.g., profit) is calculated and displayed 7. User views AI insights and product performance dashboard	- Product is successfully stored in the database - AI generates structured and meaningful summary output - KPI values (profit, trends) are correctly calculated - API responses are returned within acceptable time (< 800ms) - Dashboard displays correct and consistent data	- Product successfully stored - AI summary generated and formatted correctly - KPI values calculated accurately - API response time recorded at ~450ms - Dashboard reflects correct insights - Status: Passed
TC-02	**Happy Case (Feature Flow):** Product Update → AI Re-analysis	Verify that when a user updates an existing product's data (e.g., price or category), the system correctly	1. Login to the system with valid credentials 2. Navigate to product management	- Product details are successfully updated in the database	- Product updated successfully - AI analysis re-generated based on new

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
		saves the changes, triggers AI re-analysis, and updates KPI results accordingly.	page 3. Select an existing product 4. Update product details (e.g., change selling price or category) 5. Save the updated product 6. Trigger AI analysis for the updated product 7. View updated KPI metrics and AI-generated insights	- AI analysis reflects the updated product data - KPI values (e.g., profit) are recalculated correctly - Updated insights are displayed on the dashboard - No data inconsistency between old and new values	data - KPI values updated correctly - Dashboard reflects latest insights without error
TC-03	**Specific Case (Negative):** Invalid AI Input / Data Validation	Verify that the system handles invalid or incomplete input data properly when triggering AI analysis, without causing system crashes or unhandled exceptions.	1. Login to the system 2. Attempt to add a product with missing or invalid fields (e.g., null price or negative cost) 3. Trigger AI analysis on incomplete dataset 4. Observe system response 5.	- System blocks invalid input submission - Proper validation error message displayed (e.g., “Invalid product data”) - AI analysis is not executed on invalid data - System remains stable (no 500 server error) - User is prompted to correct input	Invalid data is blocked No clear error message displayed to user System remains stable without crashing User feedback is insufficient Status: Failed

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-04	**Specific Case (Negative):** Database Retrieval Failure	Verify that the system handles database failure or data retrieval issues gracefully when fetching product or AI summary data, without crashing or exposing internal errors.	1. Login to the system 2. Temporarily simulate database connection failure 3. Attempt to fetch product list or AI summary data 4. Observe system response	- System displays user-friendly error message (e.g., Unable to retrieve data at the moment. Please try again later.) - No raw SQL/stack trace error is exposed API returns proper error status (e.g., 500 with structured JSON response) - System remains stable and does not crash	- API returns error but exposes technical message - No user-friendly message displayed - System remains running but error handling is not properly implemented
TC-05	**NFR (Performance):** API Response Test	Verify that the system maintains acceptable API response time and AI processing performance under concurrent usage.	1. Use Postman or load testing tool 2. Send 50 concurrent requests to AI analysis endpoint 3. Measure average API response time 4. Record error rate	- Average response time < 800ms - AI processing completes within acceptable delay - Error rate = 0%	- Average response time recorded at 1128ms - No errors detected (0% error rate) - Response time exceeds acceptable threshold Status: Failed
TC-06	**NFR (Load):** Database & KPI Processing Under Load	Verify system stability and performance when handling large volumes of product	1. Insert large dataset (e.g., thousands of product records and reviews)	- System handles large dataset without crashing - KPI	- System remains stable - KPI calculations

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
		and review data for KPI computation.	2. Trigger KPI calculation and AI summary generation 3. Monitor system performance and processing time	calculations remain accurate - Processing completes within acceptable time - No database timeout or failure	are correct - Slight delay observed in processing large dataset - No system crash or failure

6. AI Output & Boundary Testing (Drafts)

This section evaluates the reliability, correctness, and robustness of the AI (GLM) integration within the Intelligent E-commerce System - *EZ System*. The testing ensures that AI-generated outputs such as product insights, summaries, and KPI interpretations are accurate, relevant, and safe under both normal and abnormal input conditions.

6.1. **Prompt/Response Test Pairs**

The following test cases validate whether the AI produces meaningful and accurate outputs based on business-related prompts such as product analysis and review summarization.

Test ID	Prompt Input (Simplified System Input)	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	Product with high sales, high profit, high conversion rate + rule-based positive insights	AI should identify strong performance, confirm business success, and recommend scaling strategy. Output must be valid JSON with keys: `recommendation`, `trade_off_analysis`, `impact_analysis`. No fabricated metrics allowed.	AI correctly identified strong performance, recommended expansion and inventory scaling. JSON format valid.	Pass

AI-02	Product with low conversion rate, low rating, high CPI value + negative rule-based insights	AI should detect poor performance, highlight inefficiency, and recommend cost reduction or marketing optimization. Must not contradict provided KPI values.	AI correctly identified low conversion efficiency and suggested pricing + marketing improvements. JSON output valid.	Pass
AI-03	Balanced product (moderate sales, moderate profit, mixed CPI/PPI signals)	AI should provide balanced trade-off analysis between cost and profit, and suggest optimization strategies. Must reflect KPI trade-offs accurately.	AI generated balanced recommendations and correctly analyzed trade-offs between cost and performance.	Pass
AI-04	Product with inconsistent KPI input (e.g., high sales but low profit, high CPI)	AI must detect inconsistency and highlight possible inefficiency in trade-offs. Must NOT assume missing values or hallucinate data.	AI flagged inefficiency between sales and profit and suggested a cost optimization strategy.	Pass

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	1024 tokens (configured in `llm_engine` for AI analysis endpoint)
Input Used While Testing	1200 tokens (exceeding threshold deliberately)
Expected Behavior	Reject input OR chunk into segments
Actual Behavior	System accepted input and returned output beyond threshold
Test Output	AI response generated but not truncated

Status	❌ Fail — system did not enforce maximum input size

6.3. Adversarial/Edge Prompt Test

Test ID	Input Scenario (row data)	Expected Behavior (Acceptance Criteria)	Actual Behavior	Status
AI-05	conversion_rate: NULL, total_sales: 500, total_revenue: NULL, estimated_profit: NULL, avg_rating: 4.0, CPI/PPI: NULL	The system must safely handle missing KPI values without crashing. rule_engine() should process only valid fields. No hallucination allowed. llm_engine() must fallback to rule_output if needed. Output must remain valid JSON.	System executed successfully with conditional checks. Partial insights generated (views vs cart issue detected). Fallback mode works when AI is disabled.	Pass
AI-06	total_sales: 100, conversion_rate: 0.01, other KPI fields NULL, rule_output: partial	System must not hallucinate missing data. Only applicable rules should trigger (low conversion). AI must return fallback or limited analysis response. JSON must remain valid.	Rule engine correctly detected low conversion issue. Limited analysis generated. Fallback mode ensured stability. No crash observed.	Pass

6.4. Hallucinating Handling

1. Hallucination Risk Sources

- Hallucination may occur when KPI fields such as sales, revenue, profit, or conversion rate are missing or contain null values.
- It can also happen when the `rule_output` generated by the rule engine is incomplete or partially available.
- Inconsistent business data, such as high sales but negative profit without clear supporting context, may also trigger incorrect AI assumptions.
- The AI model may attempt to infer or guess missing values if input validation is weak or insufficient.

2. Structured Input Control

- The system enforces structured input using predefined KPI variables including sales, revenue, profit, conversion rate, rating, CPI, and PPI.
- Only validated and structured data from the backend is passed into the GLM model.
- This reduces ambiguity and prevents the AI from processing unstructured or irrelevant inputs.

3. Rule-Based Preprocessing Layer

- The `rule_engine()` function generates deterministic business insights before AI processing.
- Business conditions such as low conversion rate, negative profit, or low rating are handled using predefined logic.
- This ensures that essential insights are computed without relying on AI inference.
- The rule-based output (`rule_output`) provides grounded context for the AI model.

4. Controlled Prompt Design

- The AI prompt strictly defines the task scope, including performance explanation, recommendation generation, trade-off analysis, and trend prediction.
- The model is constrained to use only the provided KPI data and rule-based insights.
- The output format is restricted to a structured JSON format with predefined fields.
- This prevents unstructured responses and reduces hallucination risk.

5. Fallback Mechanism (Safety Layer)

- If the AI API key is not available, the system automatically switches to rule-based output only.
- This fallback mechanism ensures that no AI-generated hallucination occurs in the absence of the model.
- The system remains stable and continues to provide deterministic business insights.

6. Output Validation (Recommended Improvement)

- The system should validate that AI responses are in correct JSON format before saving or displaying results.
- Required fields such as recommendation, trade_off_analysis, and impact_analysis must always be present.
- Invalid or incomplete outputs should be rejected and replaced with fallback rule-based results.

7. Overall Outcome

- The system significantly reduces hallucination risk through structured data input and rule-based preprocessing.
- The AI model is constrained to generate responses only within the defined business context.

- However, edge cases with missing or incomplete data still require further validation improvements.
- Additional enhancements such as input validation layers and confidence scoring are recommended for stronger hallucination control.